

Memo: Where We Stand on `/kb-regenerate`

Date: 2026-04-29 **Author:** Black (with Claude) **Audience:** EAGLE team **TL;DR:** The `/kb-regenerate` command exists, works, and is the only thing keeping our KB metadata, checklists, templates, and compliance matrix from silently drifting. It is **not wired into CI** — it only runs when a human invokes the slash command in Claude Code. There is also a bigger problem behind it: KB content authority is split across three buckets (`rh-eagle-files` , `eagle-documents-.../eagle-knowledge-base/` , and the S3 Vectors index `rh-eagle/eagle-kb-approved`), with a measured ~84-doc drift between the source and the dest bucket and **no production runtime code that writes vectors**. We need to (a) close the immediate drift, (b) extend `/kb-regenerate` to validate the vector-side of that pipeline, and (c) decide whether vectors become the canonical retrieval lane.

Update — same day, 2026-04-29 PM (post-write): Several memo claims have already been overtaken by work in progress on `fix/semantic-lane-iam-titan-embed`. (1) The semantic lane wasn't just stale — it was returning `[]` on every call due to a missing `bedrock:InvokeModel` grant on `amazon.titan-embed-text-v2`; fixed in commit `f41847a` (15:06 EDT). (2) A new env-scoped vectors bucket name has been chosen — `eagle-kb-vectors-{account}-{env}` — wired into `environments.ts` and the ECS task env (`S3_VECTORS_BUCKET` / `S3_VECTORS_INDEX`); bucket is created **manually via AWS CLI** because S3 Vectors lacks CDK L2 constructs. (3) `storage-stack.ts` now adds an explicit `DenyKnowledgeBaseWrites` IAM statement — runtime agent can never `Put / Delete` under `eagle-knowledge-base/*`, codifying KB ingestion as Lambda + operator-script only. (4) `scripts/migrate-knowledge-base.py` is moving to `scripts/archive/migrate-knowledge-base.py` (not lost). (5) Separate retrieval-quality fix in flight: `_FORCED_AGENT_ROUTES` + keyword expansion in `strands_agentic_service.py` to address the Q4/Q5 Part-15 misroute on SBIR / IDIQ-Part-16 queries. Treat the open architectural decisions below as partially resolved — sections #1 (canonical bucket) and #3 (runtime never writes) are decided.

What it is

A Claude Code slash command at `.claude/commands/kb-regenerate.md`, introduced in commit **a522b2d** (2026-04-03) and last updated in **eff6b45** (2026-04-27, Alvee's KB v3 path fix). It runs up to five phases:

Phase	Purpose	What it touches
1. Orphan purge	Scan <code>eagle-document-metadata-dev</code> , head-check each <code>s3_key</code> , delete entries whose S3 object is gone	DynamoDB
2. Checklist + template validation	<code>head_object</code> every checklist (9) and template (11) the backend hardcodes against	S3 only — read
3. Compliance matrix deep analysis	<code>scripts/kb_matrix_analysis.py</code> — three layers: <code>matrix.json</code> internal consistency, backend <code>compliance_matrix.py</code> sync, frontend <code>matrix-data.ts</code> sync (Ctrl+M modal)	Source files only — read
4. Metadata regeneration	Optional, behind <code>--regenerate <key></code> or <code>--regenerate-all</code> . Invokes <code>eagle-metadata-extractor-dev</code> Lambda asynchronously per S3 object	Lambda + DynamoDB
5. Summary report	Consolidated PASS/FAIL table	stdout only

Default invocation runs Phases 1–3 + 5. Phase 4 is opt-in.

How to invoke

From inside Claude Code (any session in this repo):

```
/kb-regenerate                # full hygiene run (dry orphan delete OFF – will p
/kb-regenerate --dry-run      # report only, deletes nothing
/kb-regenerate --regenerate <s3-key> # re-extract metadata for one document
/kb-regenerate --regenerate-all  # re-extract metadata for every doc under approved
```

It needs the `eagle` AWS SSO profile (us-east-1, account 695681773636 / NONPROD). The command body uses inline `python -c "..."` blocks invoked via Bash, so any environment that can run `boto3` against the dev account works.

Is it wired up?

No — not in the way "wired up" usually means.

- ❌ Not in `.github/workflows/` — no scheduled or PR-triggered run.
- ❌ Not in `Justfile` — no `just kb-regenerate` recipe.
- ❌ Not in `server/app/` — no API endpoint.
- ❌ Not in the deploy pipeline — `deploy.yml` does not gate on KB integrity.

-  Referenced in two docs: the KB v2 migration plan (docs/20260413-100000-plan-kb-v2-migration-v1.md , Phase 4 step 8) and the weekly changelog. Both treat it as a manual cleanup step.
-  The Phase 3 helper (scripts/kb_matrix_analysis.py) is a standalone Python script — runnable without Claude.

In practice that means the only way it executes today is when a developer remembers to type `/kb-regenerate` in their Claude session. That is fine for the orphan-purge phase (destructive, wants a human), but Phases 2 and 3 are pure validators and should arguably run on every PR that touches `eagle-plugin/data/matrix.json` , `compliance_matrix.py` , `template_registry.py` , `knowledge_tools.py` , or `matrix-data.ts` .

Recent commits that bear on whether the command needs enhancement

Commit	Date	Why it matters
a522b2d	2026-04-03	Created <code>/kb-regenerate</code> and <code>kb_matrix_analysis.py</code> . Established the 5-phase shape.
af83106	2026-04-02	Compliance matrix Phases 2e–6 landed: <code>_CONFIDENCE</code> , <code>_extract_far_citations</code> , <code>_verify</code> , <code>get_template_fields</code> , <code>_related_far</code> . Phase 3 of kb-regenerate does not yet check confidence-dict consistency or <code>_verify</code> output shape.
e39b567	2026-04-09 (est.)	Added new matrix keys: <code>intake_required_facts</code> , <code>budget_semantics</code> . Phase 3 layer-A does not yet validate these new top-level keys.
e12fcdd / d14a41d	mid-Apr	Tier compliance docs (core vs supplemental). New schema fields under <code>doc_rules</code> . Layer A check should grow accordingly.
7ebbf7a	mid-Apr	Semantic retrieval Lane 1e (S3 Vectors + Titan v2). Nothing in kb-regenerate validates vector index sync against <code>eagle-documents</code> bucket — when a doc is deleted from S3, its vector lingers.
eff6b45	2026-04-27	KB v3 path migration. Hardcoded paths in kb-regenerate's CHECKLISTS / TEMPLATES lists had to be hand-edited in lockstep with <code>knowledge_tools.py:BUILTIN_KB_ENTRIES</code> . This is a maintenance smell — both lists are duplicates of the same source of truth.
cf0d3bb / 8213928	2026-04-26 → 2026-04-28	Per-lane source transparency + lane breakdown modal. Adds new SSE wire-shape (<code>lane</code> , <code>score</code> , <code>lane_breakdown</code>). Surface area where matrix→frontend drift would now be user-visible.

Bigger picture: the S3 Vectors / canonical-source problem

Investigation today (separate Claude session, verified against the live AWS account) surfaced a structural issue that the current `/kb-regenerate` does not catch. KB content authority is split across **three buckets that no production code keeps in sync**:

```
rh-eagle-files                                ← manually edited, 343 objects
| (one-time copy via scripts/migrate-knowledge-base.py,
| tracked in git but currently deleted on this branch
| and known stale – 84-doc drift)
▼
eagle-documents-{acct}-{env}/                  ← 259 objects (last write 2026-04-28)
  eagle-knowledge-base/approved/
  | (read by the metadata + path retrieval lanes via
  | knowledge_tools.py; written manually only)
  ▼
rh-eagle/eagle-kb-approved (S3 Vectors index) ← STATIC. Seeded externally.
                                              Queried by Lane 1e (semantic).
                                              No in-app PutVectors anywhere.
```

Specifically: - `scripts/migrate-knowledge-base.py` (committed in `27ce3bc`, 2026-03-04) is the one-shot batch that copies `rh-eagle-files` → `eagle-documents-.../eagle-knowledge-base/`. It is currently *deleted in the working tree on `fix/semantic-lane-iam-titan-embed`* — visible via `git status` — which means anyone branching from there loses the migration tool. The drift is real: **84 docs (~25%) live on the source but not the dest.** - `server/scripts/backfill_s3_vectors.py` (committed in `664dfa7`, mid-Apr) is the matching one-shot for vectors — chunks docs, embeds via Bedrock Titan v2, upserts into `rh-eagle/eagle-kb-approved`. Idempotent, but **manual**. There is no `PutVectors` anywhere in `server/app/` runtime code. - The metadata-extraction Lambda exists at `infrastructure/cdk-eagle/lambda/metadata-extraction/` (writes DynamoDB, **not vectors**). To make vectors update automatically on doc upload, this Lambda would need an embed step bolted on.

Net effect: today, retrieval freshness on the semantic lane depends entirely on whoever last ran `backfill_s3_vectors.py`, and we don't have telemetry telling us when that was. The metadata/path lanes depend on whoever last ran `migrate-knowledge-base.py`. Both are invisible to `/kb-regenerate` Phase 1's orphan purge, which only checks DynamoDB ↔ `eagle-documents` consistency.

Architectural decisions still open

1. **Canonical source bucket.** Pick one — `eagle-knowledge-base-{account}-{env}` (env-scoped, matches our convention) or one bucket with `pending/ / approved/` prefixes mirroring the metadata-extraction Lambda's S3-event shape.
2. **Reprocess execution model.** Drop-and-trigger via S3 event → Lambda (embeds + writes vectors + moves to `approved/`), or scheduled daily re-embed job over the source bucket.

3. **Retrieval architecture.** Vectors as primary with metadata/path as fallback, or full cutover? And do agent-prompt files (`approved/agents/02-legal.txt` , fetched directly by S3 key via `knowledge_fetch`) move to vectors too, or stay direct-fetch?
4. **Migration sequence.** One-shot cutover, or run vectors alongside the existing lanes for a sprint?

These belong in a proper plan spec — not this memo — but `/kb-regenerate` enhancements should anticipate the answer.

Suggested enhancements (rough priority order)

1. **De-duplicate the checklist + template lists.** Phase 2 hardcodes 9 checklists and 11 templates. The same lists live in `server/app/tools/knowledge_tools.py` (`BUILTIN_KB_ENTRIES`) and `server/app/template_registry.py` (`TEMPLATE_REGISTRY`). Have Phase 2 import from those modules instead. KB v3 only had to touch four files because of this duplication; next migration shouldn't.
2. **Teach Phase 3 about new matrix keys.** Add Layer-A validators for `intake_required_facts` and `budget_semantics` (commit `e39b567`), and core-vs-supplemental tier fields (commit `e12fcdd`).
3. **Add a Phase 3D: `_CONFIDENCE` and `_verify shape check`.** Confirm every `doc/compliance/approval` entry in `get_requirements()` output carries a `confidence` field (Phase 6 from the matrix overhaul) and that `_verify.far_citations` is non-empty when applicable. Could be a 30-line property-style assertion.
4. **New Phase 6 — three-bucket sync audit.** This is the big one. The phase should:
5. Diff `rh-eagle-files` ↔ `eagle-documents-.../eagle-knowledge-base/approved/` and report the drift count + missing keys (today: 343 vs 259 = 84 missing).
6. Diff S3 keys ↔ S3 Vectors index entries in `rh-eagle/eagle-kb-approved` and flag (a) orphan vectors whose source doc is gone and (b) source docs with no vector entry.
7. Surface the **last-modified timestamp** of the most recent vector upsert, since today we have no signal on vector freshness.
8. All read-only — never auto-purges vectors. Bumping is a separate, opt-in action.
9. **New flag `--reembed <s3-key> / --reembed-all`.** Wraps `server/scripts/backfill_s3_vectors.py` , mirroring how Phase 4 wraps the metadata Lambda. Gives us one consistent tool for "make the indexes match the source."
10. **New flag `--migrate-source` (or wrap the existing script).** Makes `scripts/migrate-knowledge-base.py` invocable from inside `/kb-regenerate` so it stops being an undiscoverable, sometimes-deleted file. Also: get that script restored on this branch and committed back to main so it doesn't disappear again.
11. **De-duplicate the checklist + template lists.** Phase 2 hardcodes 9 checklists and 11 templates. The same lists live in `server/app/tools/knowledge_tools.py` (`BUILTIN_KB_ENTRIES`) and `server/app/template_registry.py` (`TEMPLATE_REGISTRY`). Have Phase 2 import from those modules instead. KB v3 only had to touch four files because of this duplication; next migration shouldn't.

12. **Teach Phase 3 about new matrix keys.** Add Layer-A validators for `intake_required_facts` and `budget_semantics` (commit `e39b567`), and core-vs-supplemental tier fields (commit `e12fcdd`).
13. **Add a Phase 3D: `_CONFIDENCE` and `_verify` shape check.** Confirm every `doc/compliance/approval` entry in `get_requirements()` output carries a `confidence` field (Phase 6 from the matrix overhaul) and that `_verify.far_citations` is non-empty when applicable. Could be a 30-line property-style assertion.
14. **Wire Phases 2, 3, and 6-audit into CI.** They are read-only and fast. Make a `just kb-validate` recipe that runs them, plus a GitHub Action that runs on PRs touching the trigger files. The drift number + vector-staleness number become PR comments. Phases 1, 4, 5, and 6 stay manual / opt-in.
15. **Add a `--qa / --prod` flag.** Hardcoded constants reference `eagle-document-metadata-dev` and `eagle-documents-695681773636-dev`. QA and prod accounts will need their own values; today the command can only target dev.

Recommended next action

Two parallel tracks:

Track A — close immediate drift (today / tomorrow). 1. Restore `scripts/migrate-knowledge-base.py` on this branch (`git checkout HEAD -- scripts/migrate-knowledge-base.py`) so it isn't lost. 2. Run `/kb-regenerate --dry-run` against dev to baseline orphan count and surface any KB v3 path issues we missed. 3. Run `scripts/migrate-knowledge-base.py` (manually for now) to close the 84-doc gap between source and dest. 4. Run `server/scripts/backfill_s3_vectors.py` to refresh the vector index against the post-migration `eagle-documents` corpus.

Track B — write the architecture pivot plan. Open `.claude/specs/{ts}-plan-vectors-as-canonical-retrieval-v1.md` capturing the four open architectural decisions above. Decide whether vectors become primary retrieval and whether the metadata-extraction Lambda absorbs an embed step, before we commit to enhancement #4 (Phase 6 audit) and #5/#6 (re-embed / migrate flags) on `/kb-regenerate`. Those enhancements need to know what the steady-state pipeline looks like.

Files referenced in this memo

Path	Why
<code>.claude/commands/kb-regenerate.md</code>	The command itself
<code>scripts/kb_matrix_analysis.py</code>	Phase 3 helper
<code>server/app/compliance_matrix.py</code>	Backend matrix code (Layer B in Phase 3)
<code>server/app/tools/knowledge_tools.py</code>	<code>BUILTIN_KB_ENTRIES</code> source of truth for checklists
<code>server/app/template_registry.py</code>	<code>TEMPLATE_REGISTRY</code> source of truth for templates

Path	Why
<code>eagle-plugin/data/matrix.json</code>	Compliance matrix source of truth (Layer A in Phase 3)
<code>client/components/contract-matrix/matrix-data.ts</code>	Frontend mirror (Layer C in Phase 3)
<code>scripts/migrate-knowledge-base.py</code>	One-shot copy from <code>rh-eagle-files</code> → <code>eagle-documents-.../eagle-knowledge-base/</code> . Tracked in <code>27ce3bc</code> , currently deleted on <code>fix/semantic-lane-iam-titan-embed</code> .
<code>server/scripts/backfill_s3_vectors.py</code>	One-shot vector backfill (Bedrock Titan Embed v2 → <code>rh-eagle/eagle-kb-approved</code>). Idempotent. Manual.
<code>infrastructure/cdk-eagle/lambda/metadata-extraction/handler.py</code>	Existing Lambda — writes DynamoDB metadata, does not embed . Candidate to absorb the embed step.
<code>docs/20260413-100000-plan-kb-v2-migration-v1.md</code>	Earlier migration plan that calls <code>/kb-regenerate</code> as Step 8